

Ejercicio Nro: 13 Desarrollo de Aplicación Bancaria con TDD

Enunciado:

Tu tarea es desarrollar una aplicación informática utilizando la técnica TDD para gestionar una cuenta bancaria. La aplicación debe permitir a los usuarios abrir una cuenta, realizar depósitos, hacer retiros y transferir fondos entre cuentas. A continuación se detallan las etapas de desarrollo utilizando TDD:

Etapas 1: Especificación y prueba inicial

1. Especifica los requisitos básicos del sistema y las funcionalidades clave, como la apertura de cuenta, depósito de fondos, retiro de fondos y transferencia de fondos.
2. Escribe una prueba inicial que verifique si el sistema puede crear una instancia de una cuenta bancaria y obtener su saldo inicial.

Etapas 2: Desarrollo de las funcionalidades básicas

3. Implementa la funcionalidad para abrir una cuenta bancaria, asegurándote de que se cumplan los requisitos especificados. Ejecuta la prueba y verifica que pase correctamente.
4. Implementa la funcionalidad para realizar depósitos en una cuenta bancaria. Ejecuta las pruebas y verifica que pasen correctamente.
5. Implementa la funcionalidad para realizar retiros de una cuenta bancaria. Ejecuta las pruebas y verifica que pasen correctamente.
6. Implementa la funcionalidad para transferir fondos entre cuentas bancarias. Ejecuta las pruebas y verifica que pasen correctamente.

Etapas 3: Pruebas adicionales y mejoras

7. Escribe pruebas adicionales para cubrir casos de prueba específicos, como intentar retirar más dinero del disponible en una cuenta o transferir fondos a una cuenta inexistente.
8. Ejecuta todas las pruebas y verifica que pasen correctamente.
9. Refactoriza tu código si es necesario para mejorar su estructura, legibilidad y eficiencia.
10. Ejecuta todas las pruebas nuevamente para asegurarte de que el código refactorizado no haya introducido errores.

Etapas 4: Cobertura completa de pruebas

11. Asegúrate de que todas las funcionalidades del sistema estén cubiertas por pruebas automatizadas.

12. Examine los casos límite y situaciones excepcionales para garantizar que el sistema se comporte correctamente en todos los escenarios.
13. Ejecuta todas las pruebas y verifica que pasen correctamente.

Recuerda seguir el enfoque TDD, donde agregarás una prueba antes de implementar cada funcionalidad y verificarás que todas las pruebas pasen antes de pasar a la siguiente etapa. Esto te ayudará a desarrollar una aplicación confiable, mantenible y que cumpla con los requisitos establecidos.

Resolución:

Etapas 1: Especificación y prueba inicial

1. Especificación de requisitos básicos y functionalities clave

- **Apertura de cuenta:** Creación de una instancia de cuenta asociada a un identificador único y un titular, con la opción de asignar un saldo inicial.
- **Depósito de fondos:** Incremento del saldo disponible mediante el ingreso de un monto válido y positivo.
- **Retiro de fondos:** Reducción del saldo mediante una extracción, validando que no se supere el dinero disponible ni se ingresen montos negativos o iguales a cero.
- **Transferencia de fondos:** Traspaso de dinero desde una cuenta origen hacia una cuenta destino de forma segura y atómica.

2. Prueba inicial (Ciclo TDD - Fase Roja)

Objetivo: Validar la creación correcta de la entidad y la lectura del saldo inicial asignado antes de escribir el código de producción.

JavaScript

```
// archivo: cuentaBancaria.test.js
```

```
const assert = require('assert');
```

```
const { CuentaBancaria } = require('./cuentaBancaria');
```

```
try {
```

```
  console.log("Ejecutando Prueba Inicial: Creación de Cuenta...");
```

```
  const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);
```

```
  assert.strictEqual(cuenta.numeroCuenta, "12345");
```

```
  assert.strictEqual(cuenta.obtenerSaldo(), 500.0);
```

```
  console.log("✓ Prueba superada.");
```

```
} catch (error) {
```

```
  console.error("✗ Resultado esperado en TDD (Fase Roja): La prueba falla porque la clase aún no existe.");
```

```
  console.error(error.message);
```

```
}
```

Etapa 2: Desarrollo de las funcionalidades básicas

3. Implementación para abrir una cuenta bancaria (Fase Verde)

Escribimos el código de producción mínimo indispensable en un archivo centralizado para que la prueba inicial pase a verde.

```
JavaScript
// archivo: cuentaBancaria.js
class CuentaBancaria {
  constructor(numeroCuenta, titular, saldoInicial = 0) {
    this.numeroCuenta = numeroCuenta;
    this.titular = titular;
    this.saldo = saldoInicial >= 0 ? saldoInicial : 0;
  }

  obtenerSaldo() {
    return this.saldo;
  }
}

module.exports = { CuentaBancaria };
```

Resultado: Al ejecutar la prueba inicial con este archivo creado, el resultado pasa a **Verde** exitosamente.

4. Implementación para realizar depósitos

- **Código de la prueba (Fase Roja):**

```
JavaScript
const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);
cuenta.depositar(200.0);
assert.strictEqual(cuenta.obtenerSaldo(), 700.0);
// Error: cuenta.depositar is not a function
```

- **Código de producción (Fase Verde):** Añadimos el método a nuestra clase.

```
JavaScript
// Se agrega dentro de class CuentaBancaria
```

```

depositar(monto) {
  if (monto <= 0) {
    throw new Error("El monto a depositar debe ser mayor a cero.");
  }
  this.saldo += monto;
}

```

5. Implementación para realizar retiros

- **Código de la prueba (Fase Roja):**

```

JavaScript
const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 700.0);
cuenta.retirar(300.0);
assert.strictEqual(cuenta.obtenerSaldo(), 400.0);
// Error: cuenta.retirar is not a function

```

- **Código de producción (Fase Verde):**

```

JavaScript
// Se agrega dentro de class CuentaBancaria
retirar(monto) {
  if (monto <= 0) {
    throw new Error("El monto a retirar debe ser mayor a cero.");
  }
  if (monto > this.saldo) {
    throw new Error("Fondos insuficientes.");
  }
  this.saldo -= monto;
}

```

6. Implementación para transferir fondos entre cuentas

Para orquestrar operaciones entre múltiples cuentas de forma limpia, introducimos la entidad **Banco**.

- **Código de la prueba (Fase Roja):**

```

JavaScript
const { CuentaBancaria, Banco } = require('./cuentaBancaria');

try {
  const banco = new Banco();

```

```

const origen = new CuentaBancaria("111", "Origen", 1000);
const destino = new CuentaBancaria("222", "Destino", 200);

banco.registrarCuenta(origen);
banco.registrarCuenta(destino);
banco.transferir("111", "222", 300);

assert.strictEqual(origen.obtenerSaldo(), 700);
assert.strictEqual(destino.obtenerSaldo(), 500);
} catch(error) {
  // Error: Banco is not a constructor
}

```

- **Código de producción actualizado (Fase Verde - unificado en cuentaBancaria.js):**

```

JavaScript
class Banco {
  constructor() {
    this.cuentas = new Map();
  }

  registrarCuenta(cuenta) {
    this.cuentas.set(cuenta.numeroCuenta, cuenta);
  }

  transferir(numeroOrigen, numeroDestino, monto) {
    const cuentaOrigen = this.cuentas.get(numeroOrigen);
    const cuentaDestino = this.cuentas.get(numeroDestino);

    if (!cuentaOrigen) throw new Error("Cuenta origen no encontrada.");
    if (!cuentaDestino) throw new Error("Cuenta destino no encontrada.");

    cuentaOrigen.retirar(monto);
    cuentaDestino.depositar(monto);
  }
}

// Corregido: Exportamos ambas clases en el mismo módulo
module.exports = { CuentaBancaria, Banco };

```

Etapas 3: Pruebas adicionales y mejoras

7 y 8. Pruebas adicionales y verificación (Casos Excepcionales)

Escribimos pruebas dedicadas exclusivamente a verificar que el sistema maneje correctamente los flujos alternativos y lance los errores esperados.

JavaScript

```
// Caso Excepcional A: Intentar retirar más dinero del disponible
const cuentaTest = new CuentaBancaria("999", "Tester", 100);
assert.throws(() => {
  cuentaTest.retirar(150);
}, /Fondos insuficientes/);
```

```
// Caso Excepcional B: Transferir a una cuenta inexistente
const miBanco = new Banco();
const cuentaOrigen = new CuentaBancaria("111", "Juan", 500);
miBanco.registrarCuenta(cuentaOrigen);
```

```
assert.throws(() => {
  miBanco.transferir("111", "999-INEXISTENTE", 100);
}, /Cuenta destino no encontrada/);
```

```
console.log("✓ Pruebas de casos excepcionales superadas.");
```

9 y 10. Refactorización (Refactoring) y re-ejecución de pruebas

- **Problema detectado:** Los métodos **depositar** y **retirar** repiten manualmente la condición **monto <= 0**.
- **Acción de Refactor:** Extraemos esa validación repetida a un método privado (**#validarMontoPositivo**) para limpiar el diseño sin romper el comportamiento público.

Código Refactorizado Final de **cuentaBancaria.js:**

JavaScript

```
class CuentaBancaria {
  constructor(numeroCuenta, titular, saldoInicial = 0) {
    this.numeroCuenta = numeroCuenta;
    this.titular = titular;
    this.saldo = saldoInicial >= 0 ? saldoInicial : 0;
  }

  obtenerSaldo() {
    return this.saldo;
  }
}
```

```

// Método privado refactorizado para evitar código duplicado
#validarMontoPositivo(monto) {
  if (monto <= 0) {
    throw new Error("El monto debe ser mayor a cero.");
  }
}

depositar(monto) {
  this.#validarMontoPositivo(monto);
  this.saldo += monto;
}

retirar(monto) {
  this.#validarMontoPositivo(monto);
  if (monto > this.saldo) {
    throw new Error("Fondos insuficientes.");
  }
  this.saldo -= monto;
}
}

class Banco {
  constructor() {
    this.cuentas = new Map();
  }

  registrarCuenta(cuenta) {
    this.cuentas.set(cuenta.numeroCuenta, cuenta);
  }

  transferir(numeroOrigen, numeroDestino, monto) {
    const cuentaOrigen = this.cuentas.get(numeroOrigen);
    const cuentaDestino = this.cuentas.get(numeroDestino);

    if (!cuentaOrigen) throw new Error("Cuenta origen no encontrada.");
    if (!cuentaDestino) throw new Error("Cuenta destino no encontrada.");

    cuentaOrigen.retirar(monto);
    cuentaDestino.depositar(monto);
  }
}

module.exports = { CuentaBancaria, Banco };

```

Validación: Al volver a ejecutar todas las pruebas escritas hasta el momento, el sistema sigue funcionando en **Verde**, garantizando que el refactor no rompió nada.

Etapas 4: Cobertura completa de pruebas

11 y 12. Cobertura total y análisis de Casos Límite

Para asegurar una robustez del 100%, definimos y probamos los límites del sistema (valores frontera):

1. **Monto cero exacto (0):** El sistema debe impedir depósitos o retiros de \$0 de forma explícita.
 2. **Saldo exacto:** Si una cuenta tiene \$200 y retira exactamente \$200, debe quedar con saldo \$0 sin disparar un error de "fondos insuficientes".
- **Código de pruebas para Casos Límite:**

JavaScript

```
// Límite 1: Bloquear depósitos o retiros con valor 0
const cuentaLimite = new CuentaBancaria("444", "Ana", 200);
assert.throws(() => {
  cuentaLimite.depositar(0);
}, /El monto debe ser mayor a cero/);

assert.throws(() => {
  cuentaLimite.retirar(0);
}, /El monto debe ser mayor a cero/);

// Límite 2: Permitir extracción del total exacto y quedar en cero
cuentaLimite.retirar(200);
assert.strictEqual(cuentaLimite.obtenerSaldo(), 0);
```

13. Ejecución final de la Suite de Pruebas (Fase Verde Completa)

Para validar de forma automatizada y sin depender de dependencias externas (corrigiendo la confusión de usar comandos de Jest con asserts nativos de Node), construimos el script de ejecución final.

Para probar la aplicación completa, creamos el archivo ejecutable **run-tests.js**:

JavaScript

```
// archivo: run-tests.js
```



```

const assert = require('assert');
const { CuentaBancaria, Banco } = require('./cuentaBancaria');

console.log("=== INICIANDO SUITE DE PRUEBAS TDD COMPLETA ===");

try {
  // 1. Caso de éxito básico
  const cuenta = new CuentaBancaria("12345", "Carlos Gómez", 500.0);
  cuenta.depositar(200.0);
  cuenta.retirar(300.0);
  assert.strictEqual(cuenta.obtenerSaldo(), 400.0);

  // 2. Operaciones de Banco y transferencias
  const banco = new Banco();
  const origen = new CuentaBancaria("111", "Origen", 1000);
  const destino = new CuentaBancaria("222", "Destino", 200);
  banco.registrarCuenta(origen);
  banco.registrarCuenta(destino);
  banco.transferir("111", "222", 300);
  assert.strictEqual(origen.obtenerSaldo(), 700);
  assert.strictEqual(destino.obtenerSaldo(), 500);

  // 3. Casos Excepcionales
  assert.throws(() => { origen.retirar(9999); }, /Fondos insuficientes/);
  assert.throws(() => { banco.transferir("111", "999", 50); }, /Cuenta destino no encontrada/);

  // 4. Casos Límite
  assert.throws(() => { origen.depositar(-10); }, /El monto debe ser mayor a cero/);
  assert.throws(() => { origen.depositar(0); }, /El monto debe ser mayor a cero/);

  console.log("\n► Probando Suite Completa... ✔ OK");
  console.log("\n=====");
  console.log("🎉 ¡TODAS LAS PRUEBAS PASARON EXITOSAMENTE!");
  console.log("=====");
} catch (error) {
  console.error("❌ Ocurrió un error en la suite de pruebas:");
  console.error(error.stack);
}

```

Para correr las pruebas y verificar el resultado en consola, simplemente ejecutamos en la terminal:

Bash
node run-tests.js

Resultado en Consola:

Plaintext

=== INICIANDO SUITE DE PRUEBAS TDD COMPLETA ===

► Probando Suite Completa... ✓ OK

=====

🎉 ¡TODAS LAS PRUEBAS PASARON EXITOSAMENTE!

=====